

Dos hooks que faltaban

useReducer: el carrito con acciones y toda su lógica en un archivo. useRef: hablar con el DOM y guardar lo que no se pinta. Todo sobre la Tienda Tech.

4 h 15 · L-M-V

Retomamos donde cortamos

STEERCL

SOLO DOS CONCEPTOS NUEVOS · TODO SOBRE EL PROYECTO QUE YA TIENES

El plan de hoy

0. Reapertura

8'

el punto de corte + tareas destacadas

A. Remate de la sesión anterior

0-60'

solo lo pendiente

B. useReducer

78'

el carrito con acciones · cambiar cantidad con - y +

C. useRef

73'

foco en el buscador · foco al primer error · el aviso que se apaga solo

D. Reserva: el segundo reducer

35'

los cuatro estados del catálogo en uno solo

F. Cierre + 5 tareas

20'

push demostrado, dudas y anticipo

 **Receso de 15 min** alrededor de las 2:00 · verde = **el corazón del día** · se corta a las 3:50 donde toque

LOS DOS SIRVEN PARA LO MISMO: GUARDAR ESTADO. CAMBIA CUÁNDO CONVIENE CADA UNO

useState o useReducer

useState

Un valor suelto que cambia de una o dos maneras: el término del buscador, si el panel está abierto, un campo del formulario, un interruptor.

useReducer

Un dato que cambia de **muchas** maneras (el carrito: agregar, quitar, cambiar cantidad, vaciar) · varios datos que **cambian juntos** (productos + estado + respaldo) · o cuando la lógica merece salir del componente.

```
// hoy: tres funciones que cambian el mismo dato, cada una a su manera
function agregar(p) { setItems((a) => agregarItems(a, p)); }
function quitar(id) { setItems((a) => quitarItem(a, id)); }
function vaciar() { setItems([]); }
// ...y mañana cambiarCantidad, aplicarCupon... la lógica queda repartida
```

Con un reducer, **una sola función sabe todas las maneras** en que el carrito puede cambiar. Y como es una función pura, vive en el **dominio**: sin React, sin red, sin disco.

EL COMPONENTE NO CALCULA: SOLO DICE QUÉ PASÓ

Anatomía de un reducer

despachar(accion)

el componente:
"pasó esto"



reducir(estado, accion)

función pura:
decide el estado nuevo



estado nuevo

React repinta

```
export type AccionCarrito =
  | { tipo: 'agregar'; producto: Producto } | { tipo: 'quitar'; id: number }
  | { tipo: 'cambiarCantidad'; id: number; cantidad: number } | { tipo: 'vaciar' };

export function reducirCarrito(items, accion) {
  switch (accion.tipo) {
    case 'agregar': return agregarItems(items, accion.producto);
    // ... un caso por acción, cada uno devolviendo una lista NUEVA
  }
}
```

TypeScript trabajando: dentro de cada case solo existen los datos de esa acción (en 'quitar' hay `accion.id`, no `accion.producto`). Y si agregas una acción nueva y olvidas su caso, marca error: faltaría un camino que devuelva algo.

· **El tercer argumento:** `useReducer(reducirCarrito, [], leerCarrito)` — la función de inicio, igual que `useState(leerCarrito)`.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes de useReducer

1 • modificar el estado dentro del reducer

`items.push(...)` o `item.cantidad++` y devolver la misma lista: React compara referencias, cree que nada cambió y no repinta. Siempre `map`, `filter` o `spread`

2 • tocar el mundo desde el reducer

un `fetch`, un `localStorage` o la hora actual adentro. El reducer es puro: mismos datos, mismo resultado. Lo que toca afuera va en un efecto o un manejador

3 • usar useReducer para todo

un buscador con un solo valor no necesita acciones: es más código para lo mismo. Un valor suelto se queda en `useState`

4 • acciones sin tipar

si el tipo es `string` suelto, un error de escritura pasa desapercibido y el reducer no hace nada. Con la unión de literales, TypeScript lo marca

UN OBJETO CON UNA PROPIEDAD: CURRENT · EL MISMO EN TODOS LOS RENDERS · CAMBIARLO NO REPINTA

useRef: dos usos

1 · hablar con un elemento

`ref={campo}` en el JSX y React deja el elemento en `current`. Para lo que solo el elemento sabe hacer: **poner el foco**, medir su tamaño, reproducir un video. `campo.current?.focus()`

2 · guardar lo que no se pinta

Un valor que el código necesita entre renders pero que **nadie ve**: el identificador de un temporizador, para poder cancelarlo. `temporizador.current = window.setTimeout(...)`

estado

se muestra en pantalla · al cambiar, **repinta** · se lee en el render

ref

no se muestra · al cambiar, **no repinta** · se lee en manejadores y efectos

la regla

Si se ve, es estado. Si solo lo usa el código, puede ser un ref.

`useRef<HTMLInputElement>(null)` — arranca en null porque en el primer render el elemento aún no existe: por eso se usa dentro de un efecto, y con ?.

PARA RECONOCERLOS SIN SUFRIRLOS — NINGUNO SE CODEA

Errores frecuentes de useRef

1 · guardar en un ref algo que se muestra

lo cambias y la pantalla se queda igual, porque cambiar un ref no repinta. Si se ve, es estado

2 · leer current durante el render

en el primer render vale `null`, y React no se entera de sus cambios. El ref se lee en manejadores y efectos

3 · enfocar antes de que el elemento exista

`focus()` en el cuerpo del componente: `current` todavía es `null`. Va dentro de un efecto, que corre después de pintar

4 · cambiar el DOM por fuera de React

agregar clases o borrar nodos a mano: React repinta cuando quiere y borra esos cambios. El ref es para leer o pedirle algo al elemento, no para dibujar

HASTA DONDE LLEGAMOS HOY

Lo de hoy

reducer

recibe estado y acción, devuelve el estado nuevo. El componente solo despacha

función pura → dominio

sin React, sin red, sin disco: todo lo que le puede pasar al carrito, en un archivo

acciones tipadas

unión de literales: cada acción con sus datos, y TypeScript exige los casos

useRef y el DOM

foco en el buscador, foco al primer campo con error: accesibilidad real

useRef y los valores

el temporizador del aviso: se conserva entre renders y no repinta al cambiar

la regla

si se ve, es estado; si solo lo usa el código, puede ser un ref

CRITERIO DE ENTREGA: NPM RUN BUILD SIN ERRORES + PUSH

5 tareas

FÁCIL

Replica la clase, hasta donde llegamos, con la grabación al lado. Es LA tarea.

INTERMEDIO

El reducer del catálogo: los cuatro useState de App en un solo useReducer con sus tres acciones.

INTERMEDIO

El foco del panel: al abrir el panel del carrito, que el foco vaya a su título. Pista: ref en el h2, tabIndex={-1} y un efecto que dependa de si está abierto.

DIFÍCIL

Deshacer: un botón que devuelva el carrito a como estaba antes de la última acción. El carrito anterior en un useRef (no se muestra) y una acción `restaurar` en el reducer.

DIFÍCIL

Llévalos a tu proyecto final: tu colección con useReducer, y un useRef para una mejora de accesibilidad. Explícalo en el README: es material de sustentación.

Lo que sigue

Ya conoces **todos los hooks** del día a día

```
useState · useEffect · useContext · useReducer · useRef · useMemo  
useParams · useNavigate · useLocation ← los del Router  
useCart · useSession ← los tuyos
```

Lo que queda del bloque es **tu proyecto**: llevar esto a tu tema, terminarlo y sustentarlo. Los dos hooks de hoy dan material para la defensa: por qué tu colección usa un reducer, y qué mejora de accesibilidad resolviste con un ref.

Trae las tareas hechas y el build en cero. Nos vemos.